

ADC采集

ADC采集

- 一、学习目标
 - ADC原理
 - ADC基本参数
- 二、硬件搭建
- 三、实验步骤
 - 1. 打开SYSCONFIG配置工具
 - 2. ADC参数配置
 - 3. ADC采集通道配置
 - 4. 写入程序
 - 5. 编译
- 四、程序分析
- 五、实验现象

一、学习目标

- 了解ADC基本知识。
- 学习ADC的基本使用，将采集到的数值显示在串口助手上。

ADC原理

ADC（analog to digital converter）即模数转换器，用于将模拟信号（例如电压）转换为数字信号。模拟信号是连续变化的，而数字信号则是离散的二进制数。ADC通过采样和量化模拟信号，将其转换为数字数据，以便处理器或微控制器可以进行后续处理。按照其转换原理主要分为逐次逼近型、双积分型、电压频率转换型三种。

MSPM0G3507采用的是逐次逼近型（SAR）ADC，这是一种常见的ADC工作原理。其基本思想是通过比较模拟信号与参考电压之间的大小关系，逐步逼近输入信号的数字表示。在逐次逼近型ADC中，输入信号和参考电压会通过差分放大器进行比较，产生一个差分电压。然后，这个差分电压被输入到一个逐步逼近的数字量化器中，该量化器会逐步将其与一系列参考电压进行比较。在每个逼近阶段，量化器将输入信号与一个中间电压点进行比较，并根据比较结果选择更高或更低的参考电压作为下一个逼近阶段的参考。这个过程会持续进行，直到量化器最终逼近到一个数字输出值。

ADC基本参数

- 分辨率：
 - 分辨率表示ADC转换器输出的精度，通常以位数（bit）来衡量，如8位、10位、12位等。分辨率越高，ADC能够表示的离散数字值越多，从而提供更高的精度。
 - MSPM0G3507支持8位、10位、12位的分辨率，用户可以根据需要选择合适的分辨率进行配置。
- 采样率：
 - 采样率（也叫转换速率）表示ADC对模拟输入信号进行采样的速率，通常以每秒采样次数（samples per second, SPS）表示。它表明ADC每秒钟能够进行多少次模拟到数字的转换。

- MSPM0G3507的采样率为4Msps（每秒400万次采样），适合高频率信号采集和实时数据处理。
3. 电压基准：
- ADC的电压基准是一个参考电压，它用于与模拟输入信号进行比较，最终实现模拟信号到数字信号的转换。电压基准的准确性和稳定性对ADC的转换精度至关重要。
 - MSPM0G3507
支持三种电压基准配置：
 - 内部可配置基准电压：1.4V和2.5V的专用ADC基准电压（VREF）。
 - MCU电源电压（VDD）作为基准电压。
 - 通过VREF+和VREF-引脚提供外部基准电压。若未配置电压基准，默认使用MCU的电源电压（VDD）作为基准电压。
4. 采样范围：
- 采样范围表示ADC可以采集到的模拟输入信号的电压范围，通常与基准电压的设置密切相关。范围如下： $VREF- \leq ADC \leq VREF+$
 - VREF-：设置的基准电压负端，通常为0V。
 - VREF+：设置的基准电压正端，根据软件配置来确定。

这些参数共同决定了ADC的性能，包括其精度、响应速度和输入电压范围。

二、硬件搭建

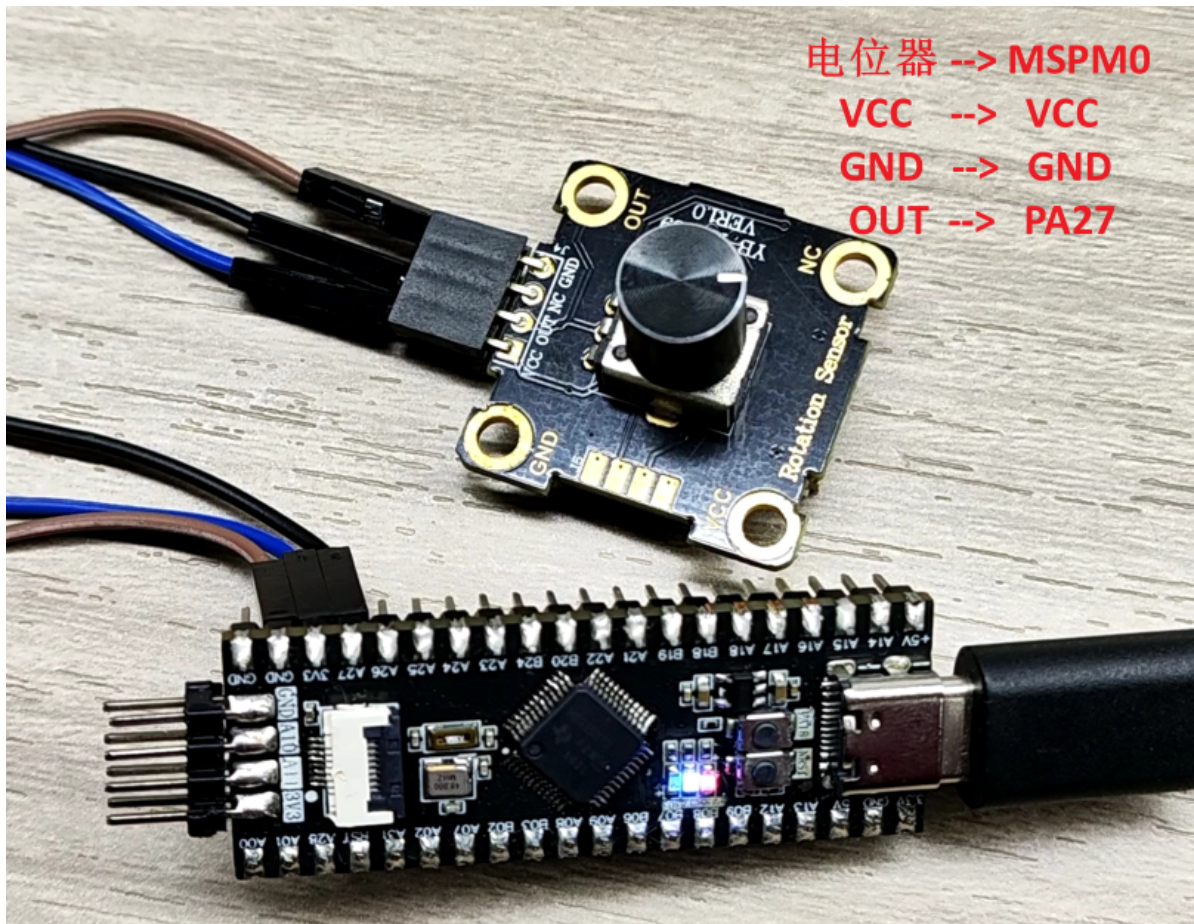
MSPM0G3507采用的是逐次逼近型的12位ADC，它有 17个多路复用通道可以转换。17个外部通道，都对应单片机的某个引脚，这个引脚不是固定的，详情请参考引脚图或者数据手册。

表 6-1. 引脚属性 (continued)						
PINCMIx	引脚名称	信号名称		引脚编号		
		模拟	数字 [引脚功能 ¹⁾]	64 LQFP	48 LQFP	32 VQFN
59	PA26	AO_1 / COMP0_IN0+ / OPA0_IN0+ / GPAMP_IN+	UART3_TX [2] / SPI1_CS0 [3] / TIMG8_C0 [4] / TIMA_FAL0 [5] / CAN_TX [6] / TIMG7_C0 [7]	30	46	30
60	PA27	AO_0 / COMP0_IN0- / OPA0_IN0-	RTC_OUT [2] / SPI1_CS1 [3] / TIMG8_C1 [4] / TIMA_FAL2 [5] / CAN_RX [6] / TIMG7_C1 [7]	31	47	31

¹⁾ 对于模拟功能，IOMUX 中的 PINCM.PF 和 PINCM.PC 应设为 0（例如，OPA 输入/输出、COMP 输入等）。器件上的每个引脚都标有数字，表示其默认功能。

功能	信号名称	引脚编号 ¹⁾				引脚类型 ²⁾	说明
		64 PM	48 PT, RGZ	32 RHB	28 DGS28		
ADC0	AO_0	31	47	31	2	I	ADC0 模拟输入 0
ADC0	AO_1	30	46	30	1	I	ADC0 模拟输入 1
ADC0	AO_2	26	45	29	28	I	ADC0 模拟输入 2
ADC0	AO_3	25	44	28	27	I	ADC0 模拟输入 3
ADC0	AO_4	27	-	-	-	I	ADC0 模拟输入 4
ADC0	AO_5	23	42	-	-	I	ADC0 模拟输入 5
ADC0	AO_6	19	41	-	-	I	ADC0 模拟输入 6
ADC0	AO_7	18	40	26	25	I	ADC0 模拟输入 7
ADC0	AO_12	7	29	18	17	I	ADC0 模拟输入 12

- 各种通道的 A/D 转换可以配置成 单次、序列转换 模式。
- 单次转换模式： 每次进行一次ADC转换后，ADC会自动停止，并将结果存储在ADC数据寄存器中。
- 重复单次转换模式： 当ADC完成一次转换后，它会自动启动另一次转换，持续的进行转换，直到外部触发或者软件触发的方式停止连续转换。
- 多通道顺序单次转换模式： 用于对多个输入通道进行依次转换。在该模式下，ADC会根据配置的通道采集顺序，对多个通道进行单次采样并转换。
- 多通道顺序重复转换模式： 用于对多个输入通道进行依次重复转换。在该模式下，ADC会根据配置的通道采集顺序，对多个通道进行重复采样并转换。
- 本案例通过ADC采集PA27引脚电压。ADC采集实验所用电位器模块及杜邦线需要另购，也可以使用其他输入电压方法。

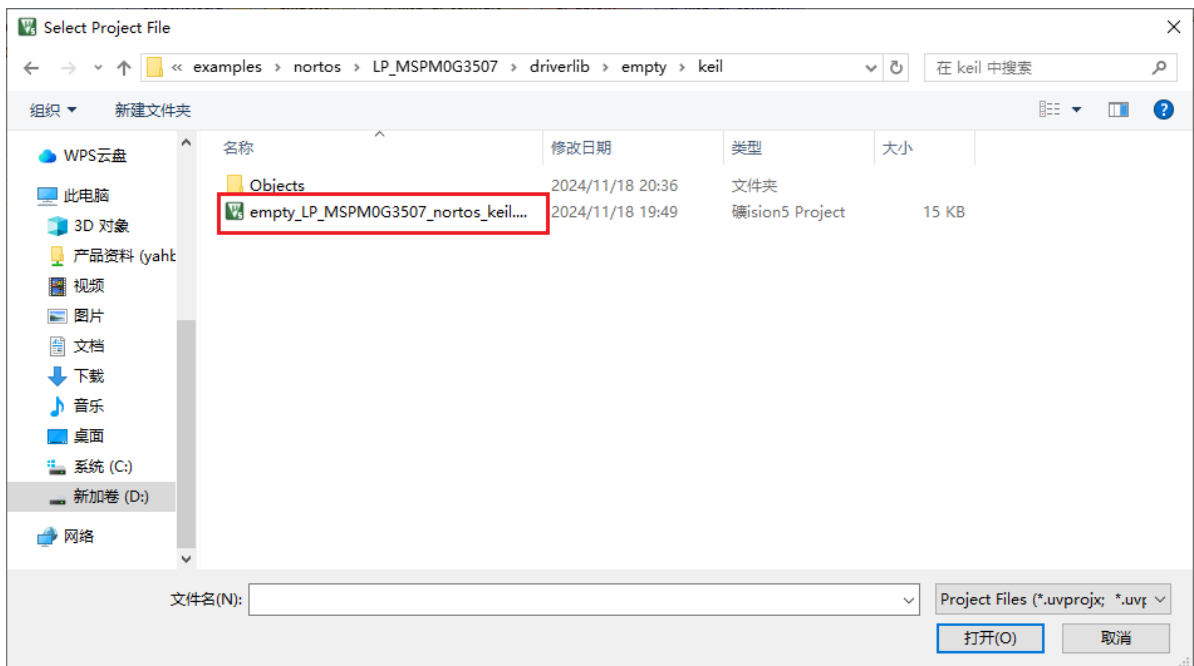


三、实验步骤

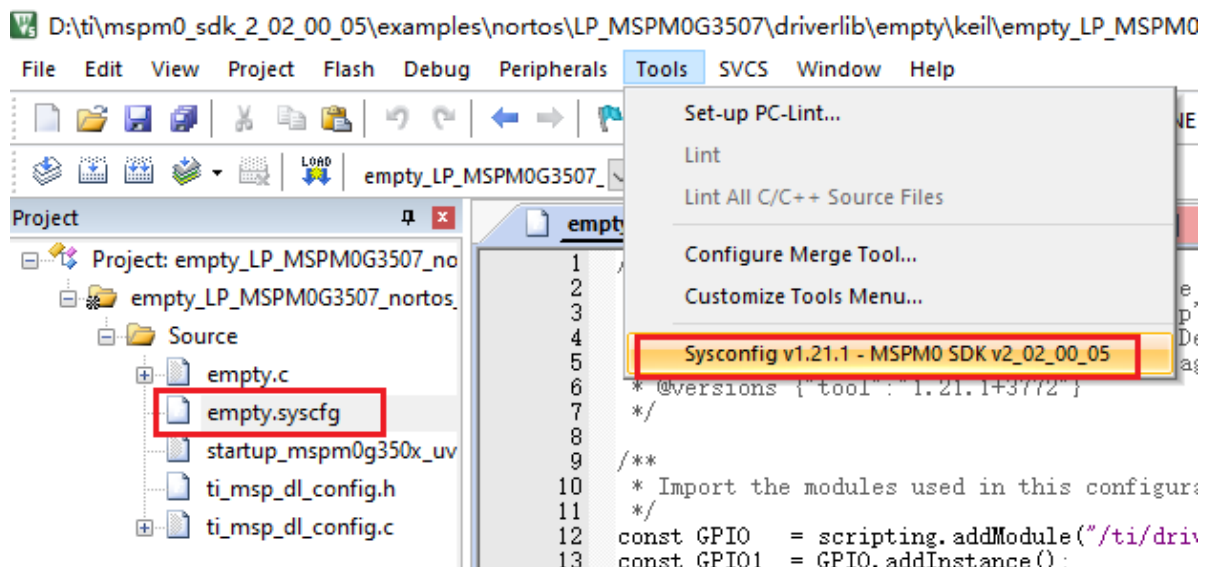
本案例将以采集PA27引脚的电压作为实验案例。通过ADC采集其电压变化，实现串口打印电压情况。

1. 打开SYSCONFIG配置工具

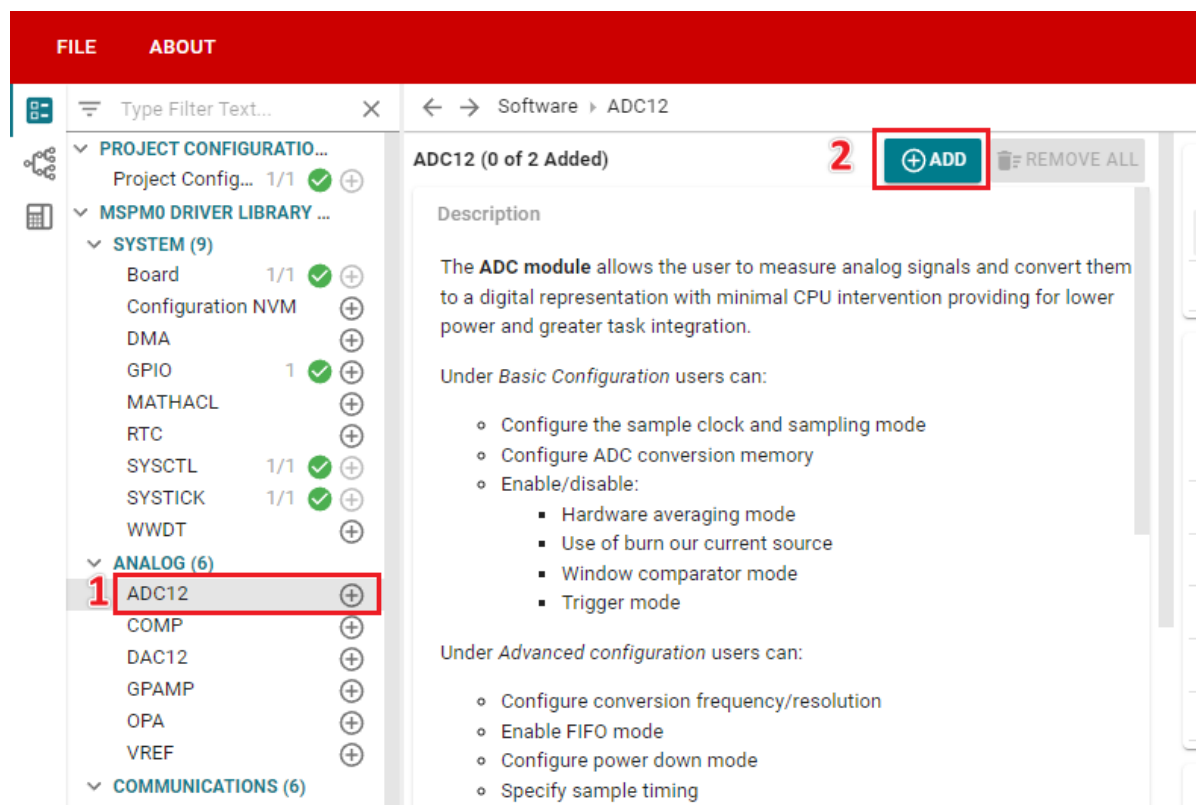
在KEIL中打开SDK中的空白工程 empty。



选中之后打开，在keil界面打开 empty.syscfg 文件，在 empty.syscfg 文件打开情况下，再在 Tools 栏选择打开 SYSCONFIG 的GUI界面。



在 SYSCONFIG 中，左侧选择 MCU 外设，找到 ADC12 选项并点击进入。在 ADC12 中点击 ADD，添加 ADC 外设。



2. ADC参数配置

串口部分的配置参考串口通讯课程，这里不再讲述。

这里选择的配置是使用32Mhz的ADC频率，采样频率为4MHz，单次重复转换模式，通过软件触发ADC开始采样，数据格式为二进制的右对齐方式。

✓

ADC_VOLTAGE

Name

ADC_VOLTAGE

Selected Peripheral

ADC0

Quick Profiles

ADC12 Profiles

Custom

Basic Configuration

Sample Clock Configuration

ADC Clock Source

SYSOSC

ADC Clock Frequency

32.00 MHz

Force SYSOSC Base Freq In STOP ®

☐

Force SYSOSC Base Freq In RUN

☐

Sample Clock Divider

Divide by 8

Calculated Sample Clock Frequency

4.00 MHz

Sampling Mode Configuration

Conversion Mode

Single

Conversion Starting Address

0

Enable Repeat Mode

☒

Sampling Mode

Auto

Trigger Source

Software

Conversion Data Format

Binary unsigned, right aligned

ADC Conversion Memory Configurations

Active Memory Control Blocks

ADC Conversion Memory 0

ADC Conversion Memory 0 Configuration

Name

ADC_CH0

Input Channel

Channel 0

Device Pin Name

PA27

Reference Voltage

VDDA

VDDA

3.30 V

Sample Period Source

Sampling Timer 0

ADC Conversion Period

10.00 ms

参数说明：

ADC Clock Source: ADC时钟来源。设置为 `SYSOSC(32MHz)`。

ADC Clock Frequency: 显示当前ADC的时钟频率。

Sample Clock Divider: 采样分频。配置为8分频。

Calculated Sample clock Frequency: 显示分频后的采样频率。

Conversion Mode: 转换模式。配置为 `Single`，单次。

Conversion Starting Address: 转换起始地址。配置从第0个开始采样(与后面的存储寄存器有关)。

Enable Repeat Mode: 是否开启重复转换。这里勾选上，开启。

Sampling Mode: 采样方式。设置为 `Auto`，自动。

Trigger Source: ADC触发方式。配置为 `Software` 软件触发方式。

Conversion Data Format: ADC数据转换格式。配置为 **Binary unsigned, right aligned** 二进制格式右对齐的方式。

MSPM0G3507的ADC支持多种数据对齐方式，以适应不同的应用场景。常见的数据对齐方式包括左对齐和右对齐。

- 右对齐模式：在右对齐模式下，ADC的数据在转换结束后被右对齐到最低位，不足的位数在高位填充0。右对齐模式允许在没有精度损失的情况下，提供更好的动态范围。

规则组数据															
0	0	0	0	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
注入组数据															
Sign	Sign	Sign	Sign	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
DAL=0															

- 左对齐模式：在左对齐模式下，ADC的数据被左对齐到最高位，不足的位数在低位填充0。左对齐模式可以提高分辨率，但会导致动态范围降低。

规则组数据															
D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	0	0	0	0
注入组数据															
Sign	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	0	0	0
DAL=1															

我们配置为右对齐方式，这样我们可以直接将转换完成的数据进行运算。

3. ADC采集通道配置

ADC Conversion Memory Configurations

Active Memory Control Blocks

ADC Conversion Memory 0

ADC Conversion Memory 0 Configuration

Name

ADC_CH0

Input Channel

Channel 0

Device Pin Name

PA27

Reference Voltage

VDDA

VDDA

3.30 V

Sample Period Source

Sampling Timer 0

ADC Conversion Period

10.00 ms

Optional Configuration

表 6-1. 引脚属性 (continued)										功能	信号名称	引脚编号 ⁽¹⁾				引脚类型 ⁽²⁾	说明
PINCmx	引脚名称	模拟	信号名称	引脚编号													
				64 LQFP	48 LQFP, VQFN	32 VQFN	28 VSSOP										
数字 [引脚功能 ⁽¹⁾]								A0_0	31	47	31	2	I	ADC0 模拟输入 0			
59	PA26	A0_1 / COMP0_IN0+ / OPA0_IN0+ / GPAMP_IN+	UART3_TX [2] / SPI1_CS0 [3] / TIMG8_C0 [4] / TIMA_FAL0 [5] / CAN_TX [6] / TIMG7_C0 [7]	30	46	30	1	A0_1	30	46	30	1	I	ADC0 模拟输入 1			
								A0_2	26	45	29	28	I	ADC0 模拟输入 2			
								A0_3	25	44	28	27	I	ADC0 模拟输入 3			
								A0_4	27	-	-	-	I	ADC0 模拟输入 4			
								A0_5	23	42	-	-	I	ADC0 模拟输入 5			
60	PA27	A0_0 / COMP0_IN0- / OPA0_IN0-	RTC_OUT [2] / SPI1_CS1 [3] / TIMG8_C1 [4] / TIMA_FAL2 [5] / CAN_RX [6] / TIMG7_C1 [7]	31	47	31	2	A0_6	19	41	-	-	I	ADC0 模拟输入 6			
								A0_7	18	40	26	25	I	ADC0 模拟输入 7			
								A0_12	7	29	18	17	I	ADC0 模拟输入 12			

1) 对于模拟功能，IOMUX 中的 PINCM_LPF 和 PINCM_PC 应设为 0 (例如，OPA 输入/输出、COMP 输入等)。器件上的每个

ADC

1) 对于模拟功能，IOMUX 中的 PINCM.PF 和 PINCM.PC 应设为 0 (例如，OPA 输入/输出、COMP 输入等)。器件上的每个

参数说明：

Active Memory Control Blocks: ADC数据存储地址。将ADC转换结果存储寄存器0。

Input Channel: 输入的通道。选择通道0（每一个通道对应不同的引脚）。

Device Pin Name: 根据通道自动选出的引脚，通道0的引脚为PA27。

Reference Voltage: 电压基准。配置为使用MCU电源VDDA。

ADC Conversion Period: 显示ADC转换单个数据的时间。

Advanced Configuration

Total Conversion Frequency

99.98 Hz

When Power Down Mode is set to Auto, ADC wakeup time may need to be considered in each sample window. Refer to the device specific data sheet for specifications on the ADC Wakeup Time.

Conversion Resolution

12-bits

Enable FIFO Mode

☐

Power Down Mode

Auto

Desired Sample Time 0

10 ms

Actual Sample Time 0

10.00 ms

Desired Sample Time 1

0 ms

Actual Sample Time 1

0.00 s

Interrupt Configuration

Enable Interrupts

MEM0 result loaded interrupt

Interrupt Priority

Default

参数说明：

Conversion Resolution: 分辨率配置为12-bits。

Desired Sample Time 0: 采样时间设置为10ms。

Enable interrupts: 开启ADC采集完成的中断。当ADC采集完成时触发中断，我们就进行数据处理。

PinMux Peripheral and Pin Configuration

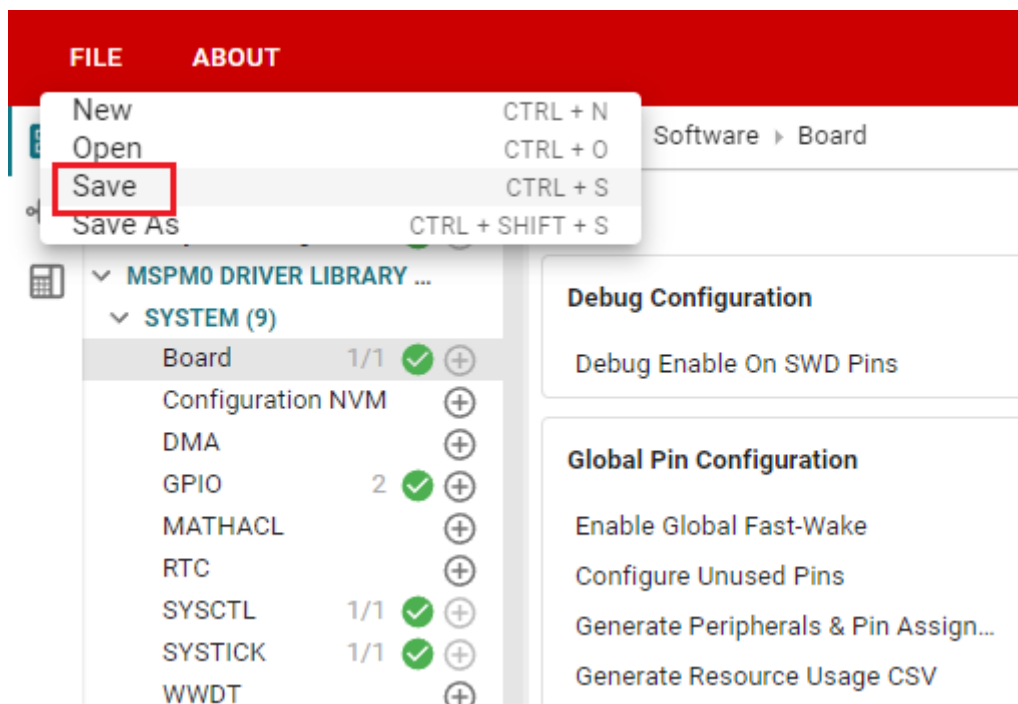
ADC12 Peripheral

ADC0

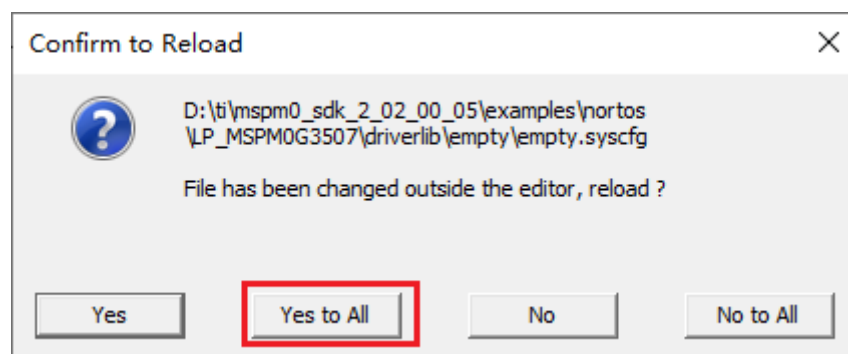
ADC12 Channel 0 Pin

PA27/31

点击 SAVE 将 SYSCONFIG 中的配置保存，之后关掉 SYSCONFIG 回到keil。



在弹窗中选择 Yes to All



同样我们还需要确认ti_msp_dl_config.c和ti_msp_dl_config.h文件是否更新。直接编译，编译就会自动更新到keil中。如果没有更新，我们也可以把 SYSCONFIG 中的文件内容复制过来。

4. 写入程序

在empty.c文件中，写入以下代码

```
#include "ti_msp_dl_config.h"
#include "stdio.h"

volatile unsigned int delay_times = 0;
volatile bool gcheckADC;          //ADC采集成功标志位  ADC acquisition success flag

void delay_ms(unsigned int ms); //搭配滴答定时器的精准毫秒级延时  Precise millisecond
delay with tick timer
unsigned int adc_getValue(void); //读取ADC的数据  Read ADC data

/*****串口重定向 serial port
redirection*****/
#if !defined(__MICROLIB)
//不使用微库的话就需要添加下面的函数
//If you don't use the micro library, you need to add the following function
#if (__ARMCLIB_VERSION <= 6000000)
//如果编译器是AC5 就定义下面这个结构体
```

```

//If the compiler is AC5, define the following structure
struct __FILE
{
    int handle;
};
#endif

FILE __stdout;

//定义_sys_exit()以避免使用半主机模式
// define _sys_exit() to avoid using semihost mode
void _sys_exit(int x)
{
    x = x;
}
#endif
//printf函数重定义 printf function redefinition
int fputc(int ch, FILE *stream)
{
    //当串口0忙的时候等待，不忙的时候再发送传进来的字符
    // wait when serial port 0 is busy, and send the incoming characters
    when it is not busy
    while( DL_UART_isBusy(UART_0_INST) == true );

    DL_UART_Main_transmitData(UART_0_INST, ch);

    return ch;
}
/*****

int main(void)
{
    unsigned int adc_value = 0;
    unsigned int voltage_value = 0;

    SYSCFG_DL_init();

    //清除串口中断标志 Clear the serial port interrupt flag
    NVIC_ClearPendingIRQ(UART_0_INST_INT_IRQN);
    //开启串口中断 Enable serial port interrupt
    NVIC_EnableIRQ(UART_0_INST_INT_IRQN);
    //开启ADC中断 Enable ADC interrupt
    NVIC_EnableIRQ(ADC_VOLTAGE_INST_INT_IRQN);

    printf("adc Demo start\r\n");
    while (1)
    {
        //获取ADC数据 Get ADC data
        adc_value = adc_getValue();
        printf("adc value:%d\r\n", adc_value);

        //将ADC采集的数据换算为电压 Convert the data collected by ADC into
        voltage
        voltage_value = (int)((adc_value/4095.0*3.3)*100);

        printf("voltage value:%d.%d\r\n",
            voltage_value/100,
            voltage_value/10%10,

```

```

        voltage_value%10 );

        delay_ms(1000);
    }
}

//延时函数 Delay function
void delay_ms(unsigned int ms)
{
    delay_times = ms;
    while( delay_times != 0 );
}

//读取ADC的数据 Read ADC data
unsigned int adc_getValue(void)
{
    unsigned int gAdcResult = 0;

    //软件触发ADC开始转换 Software triggers ADC to start conversion
    DL_ADC12_startConversion(ADC_VOLTAGE_INST);
    //如果当前状态为正在转换中则等待转换结束
    // If the current state is converting, wait for the conversion to end
    while (false == gCheckADC) {
        __WFE();
    }
    //获取数据 Get data
    gAdcResult = DL_ADC12_getMemResult(ADC_VOLTAGE_INST,
ADC_VOLTAGE_ADCMEM_ADC_CH0);

    //清除标志位 Clear flag
    gCheckADC = false;

    return gAdcResult;
}

//滴答定时器的中断服务函数 Tick ??timer interrupt service function
void SysTick_Handler(void)
{
    if( delay_times != 0 )
    {
        delay_times--;
    }
}

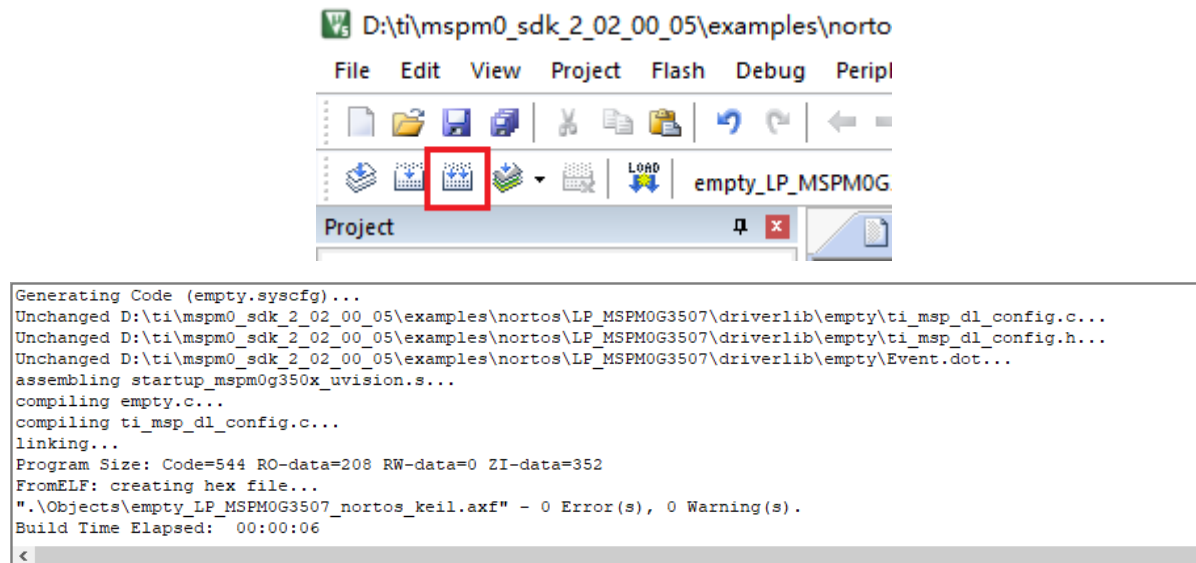
//ADC中断服务函数 ADC interrupt service function
void ADC_VOLTAGE_INST_IRQHandler(void)
{
    //查询并清除ADC中断 Query and clear ADC interrupt
    switch (DL_ADC12_getPendingInterrupt(ADC_VOLTAGE_INST))
    {
        //检查是否完成数据采集 Check whether data collection is
        completed
        case DL_ADC12_IIDX_MEM0_RESULT_LOADED:
            gCheckADC = true;//将标志位置1 Set the
            flag position to 1
            break;
        default:
            break;
    }
}

```

```
}
```

5. 编译

点击 Rebuild 图标。出现以下提示表示编译完成，并且没有错误。



四、程序分析

- empty.c

```
//读取ADC的数据 Read ADC data\nunsigned int adc_getValue(void)\n{\n    unsigned int gAdcResult = 0;\n\n    //软件触发ADC开始转换 Software triggers ADC to start conversion\n    DL_ADC12_startConversion(ADC_VOLTAGE_INST);\n    //如果当前状态为正在转换中则等待转换结束\n    // If the current state is converting, wait for the conversion to end\n    while (false == gCheckADC) {\n        __WFE();\n    }\n    //获取数据 Get data\n    gAdcResult = DL_ADC12_getMemResult(ADC_VOLTAGE_INST,\n    ADC_VOLTAGE_ADCMEM_ADC_CH0);\n\n    //清除标志位 Clear flag\n    gCheckADC = false;\n\n    return gAdcResult;\n}
```

调用 `DL_ADC12_startConversion` 函数开始ADC转换。如果当前ADC还在转换中，程序通过 `__WFE()` 进入低功耗模式，直到中断设置 `gCheckADC` 标志位，表示转换完成。转换完成后，调用 `DL_ADC12_getMemResult` 获取转换结果并返回

```
//滴答定时器的中断服务函数 Tick ??timer interrupt service function\nvoid sysTick_Handler(void)\n{\n
```

```

        if( delay_times != 0 )
        {
            delay_times--;
        }
    }
    //ADC中断服务函数 ADC interrupt service function
    void ADC_VOLTAGE_INST_IRQHandler(void)
    {
        //查询并清除ADC中断 Query and clear ADC interrupt
        switch (DL_ADC12_getPendingInterrupt(ADC_VOLTAGE_INST))
        {
            //检查是否完成数据采集 Check whether data collection is
            completed
            case DL_ADC12_IIDX_MEM0_RESULT_LOADED:
                gCheckADC = true; //将标志位置1 Set the
                flag position to 1
                break;
            default:
                break;
        }
    }
}

```

SysTick_Handler: 滴答定时器中断服务函数，用于实现延时。每次中断时，`delay_times` 减1，直到达到0停止延时。

ADC_VOLTAGE_INST_IRQHandler: ADC中断服务函数。当ADC转换完成后，检查ADC中断并设置标志位 `gcheckADC`，告知主程序可以读取数据。

```

int main(void)
{
    unsigned int adc_value = 0;
    unsigned int voltage_value = 0;

    SYSCFG_DL_init();

    //清除串口中断标志 Clear the serial port interrupt flag
    NVIC_ClearPendingIRQ(UART_0_INST_INT_IRQN);
    //开启串口中断 Enable serial port interrupt
    NVIC_EnableIRQ(UART_0_INST_INT_IRQN);
    //开启ADC中断 Enable ADC interrupt
    NVIC_EnableIRQ(ADC_VOLTAGE_INST_INT_IRQN);

    printf("adc Demo start\r\n");
    while (1)
    {
        //获取ADC数据 Get ADC data
        adc_value = adc_getValue();
        printf("adc value:%d\r\n", adc_value);

        //将ADC采集的数据换算为电压 Convert the data collected by ADC into
        voltage
        voltage_value = (int)((adc_value/4095.0*3.3)*100);

        printf("voltage value:%d.%d%d\r\n",
            voltage_value/100,
            voltage_value/10%10,
            voltage_value%10 );
    }
}

```

```
        delay_ms(1000);  
    }  
}
```

初始化系统配置之后启用串口和ADC的中断。在主循环中，通过调用 `adc_getValue()` 获取ADC数据，将其转换为电压值并打印。

五、实验现象

程序下载完成之后，对串口助手进行如下图配置之后，调节电位器旋钮，可以在串口助手接收窗口看到电压在0-3.3v之间实时变化的效果。

